Cloud Architecture Documentation

DevOps and Cloud Engineering on Azure

Marton Papp (RVR826)

May 8, 2026

Contents

1 System Overview

The project architecture is registered as an enterprise application, *azure-devops-2026-rvr826*. The system consists of the following major components:

Service	Name	Purpose
App Service Plan	*azure-devops-2026-appservice-plan*	Configures the location, scale and size of the used app services.
App Service	*votex-api*	Hosts the ASP.NET Core API backend
App Service	*votex-blazor*	Hosts the Blazor frontend application
Azure SQL Instance	*azure-devops-2026-sqlserver*	Configures the access restrictions of the database
Azure SQL Database	*Votex*	Stores application data

The frontend communicates with the backend API using HTTPS requests, while the backend interacts with the Azure SQL database using Entity Framework Core.

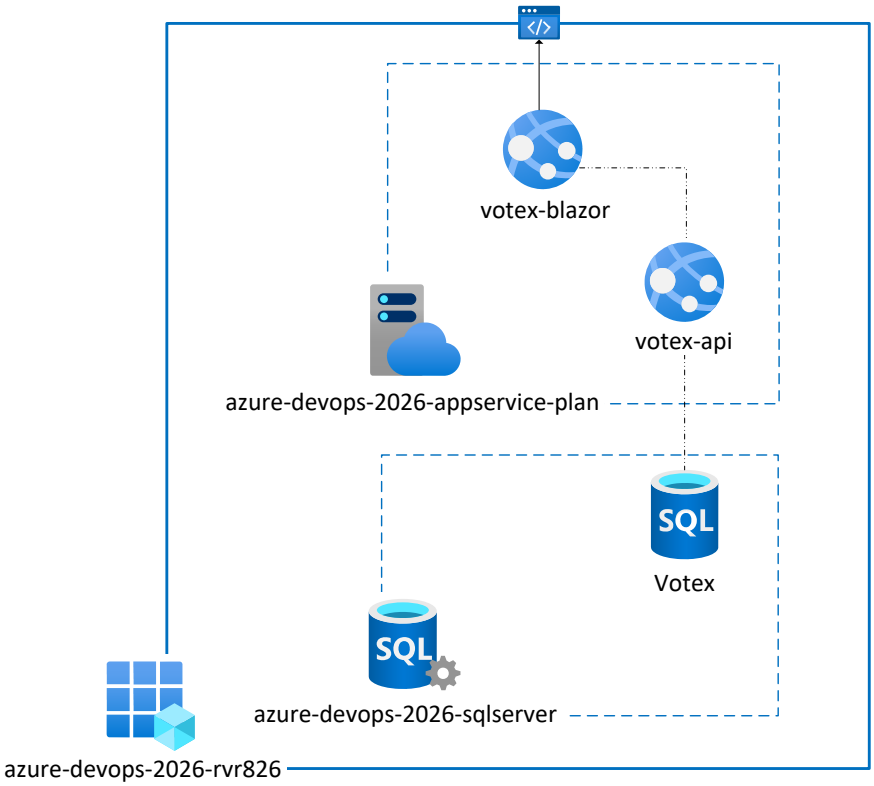

Figure 1: Cloud Architecture Diagram

1.1 Application Hosting Choice

Azure App Service was selected as the hosting platform for both the ASP.NET Core API and the Blazor frontend application. The primary reason for choosing an App Service over Virtual Machines or Kubernetes-based deployments was operational simplicity and reduced infrastructure management overhead. Using App Service provided several advantages:

- Managed hosting environment
- Built-in HTTPS support
- Integrated deployment support
- Automatic scaling capabilities
- Simplified monitoring and logging
- Native support for .NET

Alternative approaches such as Virtual Machines would require manual server configuration, additional scripting for deployment and application runtime management. Container orchestration platforms such as Kubernetes were also considered. However, Kubernetes introduces significantly higher complexity related to:

- Cluster management
- Container orchestration
- Networking configuration

For the scale and requirements of this project, Azure App Service provided the best balance between scalability, simplicity and deployment automation.

1.2 Database Hosting Choice

An Azure SQL Database was selected as the database solution instead of self-managed SQL Server deployments on Virtual Machines or containerized database instances. The decision was based on the advantages of Database-as-a-Service (DBaaS), including:

- Automatic patching
- Simplified scaling
- Reduced operational overhead
- Integrated Azure security features

Running SQL Server inside a Virtual Machine would require manual administration tasks and setup like the app host Virtual Machine counterparts.

The idea of containerized database deployments was also discarded, since the applications were not running on VMs or Kubernetes pods either, so a separate container infrastructure just for a database was not feasible.

Since the focus of the project was application and cloud deployment automation rather than infrastructure administration, Azure SQL Database provided a more suitable managed solution.

1.3 Infrastructure-as-Code Choice

Bicep was selected as the Infrastructure-as-Code solution for provisioning Azure resources. The main reasons for choosing Bicep was the native Azure integration Terraform was considered as an alternative. However, Bicep provided a more lightweight and easier solution for this project, since the cloud platform was Azure. The IaC deployment provisions:

- Azure SQL Server
- Azure SQL Database
- Network access restrictions
- App Service Plan
- Backend API App Service
- Blazor Frontend App Service

Infrastructure deployment is automated through a *GitHub Actions* CI/CD pipeline.

2 CI/CD Pipeline

2.1 Pipeline Overview

The CI/CD pipeline was implemented using GitHub Actions. The pipeline automates:

- Build process
- Automated testing
- Artifact publishing
- Infrastructure deployment
- Application deployment

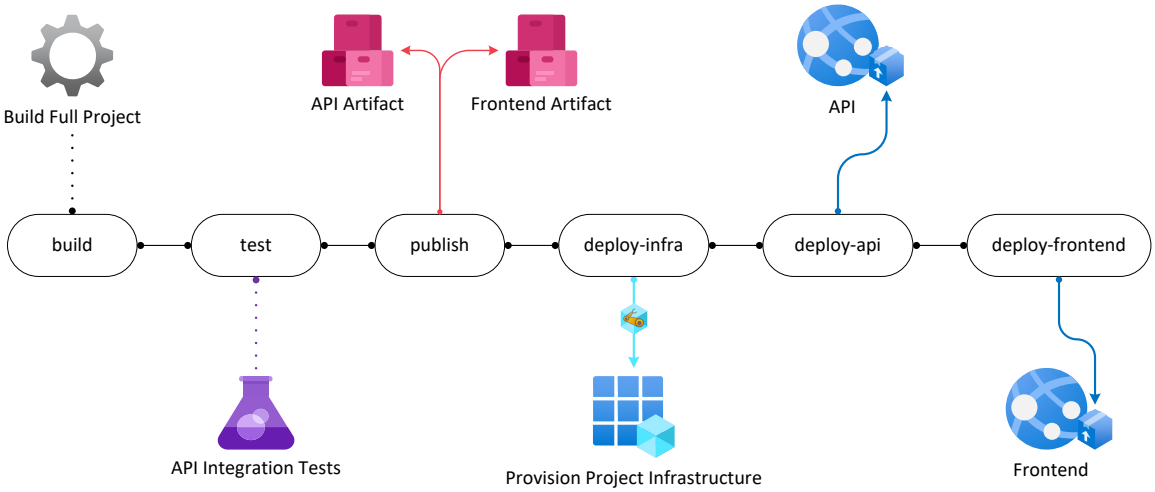

Figure 2: CI/CD Pipeline Diagram

2.2 Artifact Management

Published application outputs are stored as GitHub Actions artifacts. Deployment jobs reuse these artifacts to avoid rebuilding the applications during deployment stages.

2.3 Authentication

Authentication between GitHub Actions and Azure was configured using OpenID Connect (OIDC) federated credentials. This avoids storing Azure client secret credentials in the repository.

3 Security

3.1 JWT Authentication

The backend API uses JWT bearer authentication for securing protected endpoints. The JWT secret key value is stored securely using Azure App Service Application Settings rather than source-controlled configuration files.

3.2 CORS Configuration

CORS policies were configured to allow requests only from the deployed Blazor frontend application.

3.3 Database Security

Azure SQL firewall rules were configured to:

- Allow Azure-hosted services
- Restrict any other unauthorized external access

Database credentials are provided for the API using secure environment configuration, just like the JWT secret key.

4 Conclusion

The Votex system was successfully deployed to Microsoft Azure using Infrastructure-as-Code and automated CI/CD pipelines.

The final architecture provides:

- Automated deployments
- Scalable cloud infrastructure
- Secure configuration management
- Separation of frontend and backend services
- Centralized cloud database hosting

A Repository Structure

```
Votex.API
Votex.Blazor
Votex.DataAccess
Votex.IntegrationTests
Votex.Shared
Infrastructure/
  .github/workflows/
```

B Example Deployment Command

```
az deployment group create \
  --resource-group <RESOURCE_GROUP> \
  --template-file Infrastructure/main.bicep \
  --parameters \
    sqlServerName=<SQL_SERVER> \
    sqlAdminUser=<SQL_USER> \
    sqlAdminPassword=<SQL_PWD> \
    databaseName=<DB_NAME> \
    appServiceName=<APP_PLAN> \
    apiServiceName=<API_NAME> \
    jwtSecretKey=<JWT_SECRET> \
    frontendServiceName=<FRONTEND_NAME>
```